



COMPETITION PACK

Cloud Computing Day 3

2026 GLOBAL SKILLS CHALLENGE

WOLLONGONG, AUSTRALIA
24 - 28 June 2026



CONTENTS

CONTENTS	2
PROJECT INFORMATION	3
RULES & REGULATIONS	9
EXPERT CONTACT	9

PROJECT INFORMATION

TASK DESCRIPTION

MealMint is a recipe service that must be deployed on AWS. In this challenge, you will act as the cloud engineer responsible for designing the environment, implementing APIs, deploying applications, configuring the CI/CD pipeline, preparing the data layer, exposing the required user and API endpoints, and implementing the operational controls necessary to ensure that the service can be deployed, accessed, governed, and recovered according to the requirements described in this document.

After deploying the MealMint system, users should be able to access the application through a web browser using the CloudFront URL and perform actions such as adding recipes, viewing registered recipes, and checking ratings. In addition, according to the disaster recovery (DR) architecture, data backups should be available and discoverable in the DR Region.

For this challenge, only the requirements explicitly stated in this document will be evaluated. Therefore, you don't need to apply best practices if a requirement is not specified.

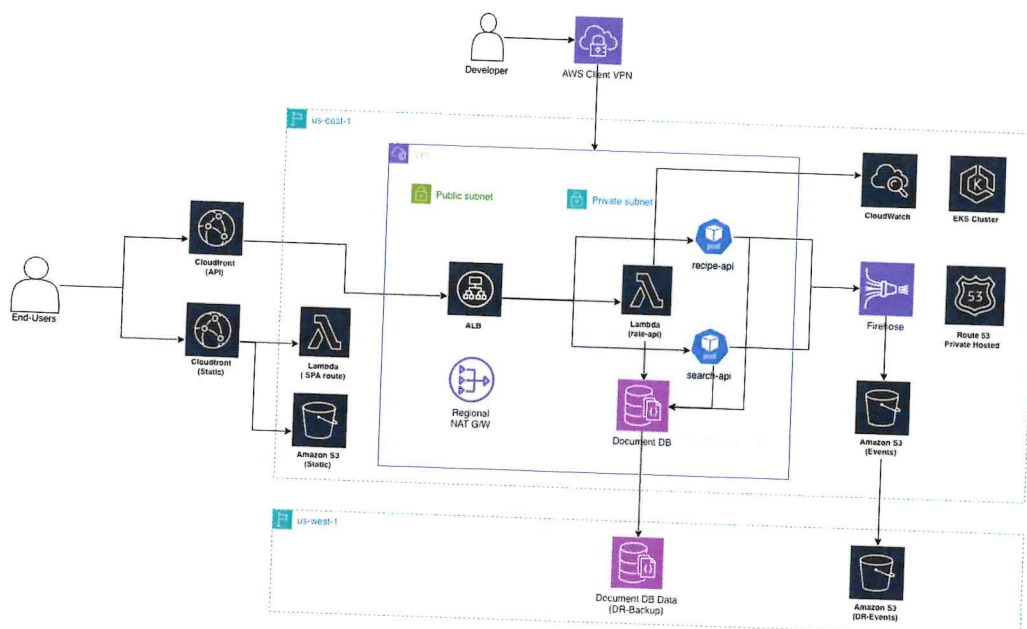
BACKGROUND

MealMint is a cloud-based service that operates in the us-east-1 region of AWS. The service provides several REST APIs for application features, and an administrator dashboard is also provided so that the service can be checked and operated through a web interface.

The company wants to modernize the system architecture by using container-based services for the main application components and serverless computing for selected functions. Application data is stored and managed in a NoSQL database so that the service can handle flexible recipe-related records and support the required API features.

To manage container deployments in a consistent way, a CI/CD workflow must be prepared. Internal operation tools such as ArgoCD should not be exposed directly to the public internet; instead, administrators must connect through VPN first and then access these private resources through the internal network.

Because the service data must be protected against regional failures, a disaster recovery strategy is also required. Important data should be copied and stored in the us-west-1 region so that the service can preserve critical information outside the main operating region.



[overall Architecture Diagram]

STAGES

Stage1: Networking

You need to create the service VPC named `mealmint-vpc` in the `us-east-1` region. The VPC must include `public` and `private subnets` across `multiple Availability Zones`, and each private subnet must be able to access the internet through a `single Regional NAT Gateway`. You also need to configure the `mealmint.internal private hosted zone`, the `mealmint-client-vpn Client VPN Endpoint` with mutual authentication, the `Mealmint-VPN local VPN profile`, and the `S3 Gateway VPC Endpoint`.

Stage2: Container Platform

You need to create the `private EKS cluster` named `mealmint-eks` and the `managed node group` named `mealmint-mng`. The container images for `recipe-api` and `search-api` must be stored in ECR repositories, and the workloads must use `EKS Pod Identity Associations`. You also need to install `ArgoCD` and use it to manage the application deployment.

Stage3: Data and Application

You need to create the `mealmint-recipes DocumentDB` cluster and import the provided seed recipe data. The `recipe-api` and `search-api` applications must be deployed to the `mealmint namespace`, and the `rate-api Lambda` function must be implemented to calculate recipe ratings from DocumentDB. You also need to create `mealmint-alb` and route the `recipe`, `search`, and `rating API` requests to the correct backend targets.

Stage4: Static Hosting and Edge

You need to upload the provided static assets to the static S3 bucket and serve them through CloudFront. The static dashboard must be configured to use the API CloudFront endpoint, and the API CloudFront distribution must forward API requests to `mealmint-alb`. Finally, two CloudFront distributions are created.

Stage5: Observability

You need to create the `mealmint-app-events` delivery stream and store application events in S3. When `config.ini` is configured, the `recipe-api` and `search-api` automatically send events to Firehose. The `rate-api Lambda` function must write application logs to Amazon CloudWatch Logs. Configure the function so that access logs are also captured and stored.

Stage6: Governance

You need to install a Kubernetes admission policy engine and apply the required workload governance policies. The policies must control container image tags and required Pod labels.

Stage7: Data Disaster Recovery

You need to configure disaster recovery for important data in a separate region. DocumentDB snapshots must be copied to the DR region, and the event data stored in S3 must be replicated to the DR bucket. Only data is required, not clusters.

CONFIGURATION DETAILS

Resource Names

The resource names and tags are used when provisioning resources.

Resource	Name
AWS Main Region	us-east-1
AWS DR region	us-west-1
VPC	mealmint-vpc
Client VPN Endpoint	mealmint-client-vpn
Local Client VPN profile	Mealmint-VPN
Private Hosted Zone domain	mealmint.internal.
ArgoCD DNS record	argocd.mealmint.internal.
EKS cluster	mealmint-eks
EKS managed node group	mealmint-mng
Kubernetes namespace – mealmint resources	mealmint
Kubernetes namespace – ArgoCD resources	argocd
ArgoCD Application - recipe	recipe-app
ArgoCD Application - search	search-app
API ALB	mealmint-alb
Lambda function – rate-api	rate-api
Lambda function – SPA rewrite	mealmint-spa-rewrite
CloudFront distribution - API	api-cf
CloudFront distribution – Static(web)	static-cf
DocumentDB cluster	mealmint-recipes
DocumentDB database	mealmint
DocumentDB collection	recipes
ECR repository – recipe	recipe-api
ECR repository – search	search-api
Static S3 bucket	mealmint-static-assets-{random_string}
Firehose S3 bucket	mealmint-events-{random_string}
DR S3 bucket	mealmint-events-dr-{random_string}
Firehose stream	mealmint-app-events
Pod Identity Association – recipe	mealmint-recipe-api
Pod Identity Association – search	mealmint-search-api
Kubernetes Deployment	recipe-api, search-api
Kubernetes ServiceAccount	recipe-api, search-api
Kubernetes Service	recipe-api, search-api

Networking

- The mealmint-vpc must include public and private subnets across at least two Availability Zones.
- Each private subnet must use the Regional NAT Gateway.
- The S3 Gateway VPC Endpoint is enabled.
- The Private Hosted Zone domain is mealmint.internal.
- The argocd.m mealmint.internal record must point to the internal ALB of ArgoCD.
- Client VPN authentication must be mutual authentication, and competitors may issue self-signed certificates if needed.
- The Client VPN profile name in your desktop must be Mealmint-VPN.
- After connecting Client VPN, it must be able to query the mealmint.internal hosted zone and access argocd.m mealmint.internal through the internal network.

EKS

- The EKS cluster type is only private mode
- It has a managed node group named mealmint-mng.
- The worker-node instance type must be t3.medium.
- The ArgoCD is deployed in the argocd namespace.
- ArgoCD must be exposed through Kubernetes Ingress using an internal ALB.
- ECR, S3 and CodeCommit are allowed for the repository of ArgoCD applications, but it may not use Github or Gitlab.
- You may install the aws-loadbalancer-controller to manage load-balancers.

Application deployment

- Application containers must use images pushed to the recipe-api and search-api ECR repositories.
- recipe-api must include Kubernetes Deployment, ServiceAccount, and Service resources.
- search-api must include Kubernetes Deployment, ServiceAccount, and Service resources.
- The recipe-api ServiceAccount must be connected to the mealmint-recipe-api Pod Identity Association.
- The search-api ServiceAccount must be connected to the mealmint-search-api Pod Identity Association.
- The search-api and recipe-api provide the /healthz API, you can invoke it after deployment.

ALB Routing

- The single ALB named mealmint-alb is created.
- The mealmint-alb is internet-facing type ALB.
- The mealmint-alb ALB must route to the following targets by condition rules.
- POST /recipes -> the recipe-api containers
- GET /recipes/search -> the search-api containers
- GET /recipes/{id}/rating -> the rate-api Lambda function

DocumentDB

- It has one primary instance and one replica instance. (The t3.medium instances are allowed)
- Backup retention must be at least seven days. (≥ 7)
- At least one manual snapshot is required.
- TLS connection is required.
- At least one cross-region snapshot copy must exist in the DR region.
- Required collection indexes are {recipeld: 1} unique, {name: 1}, and {tags: 1}.
- The provided seed file must be imported into the mealmint.recipes collection. Competitors may use the mongoimport command.
- Validation examples:
GET /recipes/search?q=thai, returns r-001 Pad Thai and r-005 Mango Sticky Rice.
GET /recipes/r-001/rating, returns rating=4.5 and count=2.
- Document schema is:

Field	Type	Description
recipeld	String	unique recipe id
name	String	recipe name
author	String	author name
tags	Array<String>	search tags
ratings	Array<Object>	used by rate-api rating calculation
createdAt	ISODate	created timestamp

CloudFront

- Two CloudFront distributions are deployed.
- It handles only HTTPS requests from clients.
- The API Cloudfront points to the mealmint-alb ALB.
- The Static CloudFront points to the mealmint-static-assets-{*} S3 bucket.

API

- All APIs must be accessible through the API CloudFront default domain.
- All APIs must respond within 10 seconds.

Path	Request Sample	Response Sample	Description
/recipes POST	{"recipeld": "z-501", "name": "Coconut", "author": "maya"}	HTTP 201, { "status": "stored", "recipeld": "z-501" }	store a new recipe
/recipes/search?q=thai GET	query string q	HTTP 200, count=2, includes Pad Thai and Mango Sticky Rice	search by name or tags
/recipes/r-001/rating GET	path parameter id	HTTP 200, {"recipeld": "r-001", "rating": 4.5, "count": 2}	ALB routes to the rate-api Lambda target, and Lambda returns the average rating from DocumentDB

Static Hosting

- Create a S3 bucket for static hosting.
- Set `apiBaseUrl` in the `config.js` file to the API CloudFront domain.
- Upload the provided static assets to the S3 bucket.
- Implement the Lambda function for Single Page
- Attach the `mealint-spa-rewrite Lambda@Edge` function to the Static CloudFront distribution.
- After configuring SPA routing, users should be able to directly access `https://<YOUR CloudFront>/recipes` and all static pages such as `/add-recipe`
- assets/`config.js` example:

```
window.MEALMINT_CONFIG = { apiBaseUrl: "https://<YOUR CLOUDFRONT ADDRESS>" };
```

Firehose and S3 DR

- Firehose stream: `mealint-app-events`.
- Firehose destination bucket: `mealint-events`.
- Firehose prefix: `events/year={!{timestamp:yyyy}}/month={!{timestamp:MM}}/day={!{timestamp:dd}}/`
- DR bucket: `mealint-events-dr`.
- It must replicate from `mealint-events` to `mealint-events-dr` using the Cross-Region-Replication feature.

Rate-API Lambda function

- Competitors must implement the `rate-api` using your preferred programming language.
- Connect to `mealint-recipes DocumentDB` cluster, and retrieve rating data from the DB.
- Use the `mealint` database and `recipes` collection.
- Find the recipe document where `recipeId` equals the path id.
- Read the ratings array and calculate the average of `ratings[].score`.
- It returns the number of rating records as `count`.
- Deploy the function in the VPC.
- The function is exposed through the ALB.
- It saves log to a CloudWatch Logs.
- Request: Parse the `recipeId` from the request path.
`GET, /recipes/<recipeId>/rating`
- Response: Respond a JSON data with `recipeId`, `rating` and `count`.
`HTTP 200, {"recipeId": "<recipeId>", "rating": <x.x>, "count": <x>}`

Governance

- Pod must not be allowed to run if any container image contains `"dev-"`.
- Pod must not be allowed to run if they have the label `"mealint/dev: dev"`.
- Before the end of competition, push a container image with a tag starting with `"dev-"` to both ECR repos.

Provided files

- Golang Binaries: `recipe-api`, `search-api` and `config.ini` (for the intel chip)
- Static assets: `index.html`, `assets/app.js`, `assets/config.js`, `assets/styles.css`
- Seed data file: `seed_recipes.json`

RULES & REGULATIONS

The competition will run to WorldSkills International Rules and Code of Ethics, Work Health and Safety is to Australian Standards and you can speak to your Australian Expert for further information. Refer to Global Skills Website

EXPERT CONTACT

Skill expert details are as follows:

NAME: Dr. Elankayer Sithirasenan

EMAIL: elankayer.sithirasenan@tafeqld.edu.au

This test project was developed by the Korean Expert - Juhyeok Bae

